UNIVERSITY OF WAH (UOW)

DEPARTMENT OF COMPUTER SCIENCE

BSCS 3RD B

REG #: UW-24-CS-BS-088

SUBMITTED BY: SAADI SAFDAR

SUBMITTED TO: Ms. MUNNIBA KHAN

COURSE TITLE: DATA STRUCTURES

COURSE CODE: CS – 201 L

LAB REPORT # 11

Task assigned in Lab: Priority Queue in Ascending Order

```
#include <iostream>
using namespace std;

class Node {
public:
    int priority, data;
    Node *next;
};

Node *front = NULL;

void insert(int item, int priority)
{
    Node *temp;
    Node *newNode = new Node();
    newNode->data = item;
    newNode->priority = priority;
    newNode->next = NULL;

    if (front == NULL || priority < front->priority)
    {
        newNode->next = front;
        front = newNode;
    }
    else
    {
        temp = front;
        while (temp->next != NULL && temp->next->priority <= priority)
        {
```

```
            temp = temp->next;
        }
        newNode->next = temp->next;
        temp->next = newNode;
    }
}

void dequeue()
{
    if (front == NULL)
    {
        cout << "Queue Underflow" << endl;
    }
    else
    {
        Node *temp = front;
        cout << "Deleted item is: " << temp->data << endl;
        front = front->next;
        free(temp);
    }
}

void display()
{
            Node *temp;
            temp = front;
            if ( front == NULL )
                        cout << "Queue is empty"<< endl;
            else
            {
```

```
                cout<<"Queu is : \n";
                cout<<"Priority item\n";
                while (temp!=NULL)
                {
                        cout<<temp->priority<<"                "<<temp->data<<endl;
                        temp = temp->next;
                }
        }
}

int main()
{
    int choice, item, priority;

    do
    {
        cout << "\n1. Insert";
        cout << "\n2. Delete";
        cout << "\n3. Display";
        cout << "\n4. Quit";
        cout << "\nEnter your choice: ";
        cin >> choice;

        switch (choice)
        {
        case 1:
            cout << "Enter data: ";
            cin >> item;
            cout << "Enter priority: ";
            cin >> priority;
```

```
            insert(item, priority);
            break;

        case 2:
            dequeue();
            break;

        case 3:
            display();
            break;

        case 4:
            cout << "Exiting program..." << endl;
            break;

        default:
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

Output

```
1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 1
Enter data: 40
Enter priority: 1

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 1
Enter data: 30
Enter priority: 2

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 1
Enter data: 20
Enter priority: 3

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 3
Queu is :
Priority item
1       40
2       30
3       20

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice:
1
Enter data: 50
Enter priority: 1

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 3
```

→

```
1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 3
Queu is :
Priority item
1       40
1       50
2       30
3       20

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 2
Deleted item is: 40

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 3
Queu is :
Priority item
1       50
2       30
3       20

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 4
Exiting program...


=== Code Execution Successful ===
```

Priority Queue in Descending Order

```
#include <iostream>
using namespace std;

class Node {
public:
    int priority, data;
    Node *next;
};

Node *front = NULL;

void insert(int item, int priority)
{
    Node *temp;
    Node *newNode = new Node();
    newNode->data = item;
    newNode->priority = priority;
    newNode->next = NULL;

    if (front == NULL || priority > front->priority)
    {
        newNode->next = front;
        front = newNode;
    }
    else
    {
        temp = front;
        while (temp->next != NULL && temp->next->priority >= priority)
```

```
        {
            temp = temp->next;
        }
        newNode->next = temp->next;
        temp->next = newNode;
    }
}

void dequeue()
{
    if (front == NULL)
    {
        cout << "Queue Underflow" << endl;
    }
    else
    {
        Node *temp = front;
        cout << "Deleted item is: " << temp->data << endl;
        front = front->next;
        free(temp);
    }
}

void display()
{
            Node *temp;
            temp = front;
            if ( front == NULL )
                        cout << "Queue is empty"<< endl;
            else
```

```
        {
                cout<<"Queu is : \n";
                cout<<"Priority item\n";
                while (temp!=NULL)
                {
                        cout<<temp->priority<<"                    "<<temp->data<<endl;
                        temp = temp->next;
                }
        }
}

int main()
{
    int choice, item, priority;

    do
    {
        cout << "\n1. Insert";
        cout << "\n2. Delete";
        cout << "\n3. Display";
        cout << "\n4. Quit";
        cout << "\nEnter your choice: ";
        cin >> choice;

        switch (choice)
        {
        case 1:
            cout << "Enter data: ";
            cin >> item;
            cout << "Enter priority: ";
```

```
            cin >> priority;
            insert(item, priority);
            break;

        case 2:
            dequeue();
            break;

        case 3:
            display();
            break;

        case 4:
            cout << "Exiting program..." << endl;
            break;

        default:
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

Output

```
1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 1
Enter data: 60
Enter priority: 1

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 1
Enter data: 70
Enter priority: 2

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 1
Enter data: 80
Enter priority: 3

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 3
Queu is :
Priority item
3       80
2       70
1       60

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 1
Enter data: 90
Enter priority: 3

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 3
Queu is :
```

→

```
1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 3
Queu is :
Priority item
3       80
3       90
2       70
1       60

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 2
Deleted item is: 80

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice:
3
Queu is :
Priority item
3       90
2       70
1       60

1. Insert
2. Delete
3. Display
4. Quit
Enter your choice: 4
Exiting program...


=== Code Execution Successful ===
```

Research on Priority Queue Applications

1. CPU Scheduling

In an operating system, CPU scheduling is the process of deciding which process in the "ready queue" gets to execute next. Using a priority queue ensures that critical tasks are completed faster than background tasks.

- **Logic:** Each process is assigned a priority value. In a **Descending Priority Queue**, the process with the highest numerical priority value is selected first.
- **Simple Examples:**
    - **System Alerts:** Handling a "Low Battery" warning before a background app update.
    - **Video Calls:** Prioritizing Zoom audio so your voice doesn't crack while an email is downloading.
    - **User Input:** Processing your mouse clicks immediately so the computer feels fast.

2. Dijkstra's Algorithm

Dijkstra's algorithm is a famous graph traversal method used to find the shortest path between a starting node and all other nodes in a weighted graph.

- **Logic:** The algorithm uses an **Ascending Priority Queue**. It stores nodes based on distance and always extracts the node with the **minimum** distance to visit next.
- **Simple Examples:**
    - **Google Maps:** Always checking the quickest street first to find the fastest way home.
    - **Internet Routing:** Finding the fastest path for a message to travel across global servers.
    - **Game AI:** Calculating the shortest walking path for a character to avoid a wall.

3. Event-Driven Simulation

This is a technique used to model the operation of a system as a sequence of events happening at specific times.

- **Logic:** Each event is assigned a **timestamp** (priority). A priority queue processes the event with the earliest time next, moving the simulation "clock" forward.
- **Simple Examples:**
    - **Hospital Triage:** Simulating an ER where a heart patient is seen before someone with a cold.
    - **Bank Lines:** Predicting how many tellers are needed based on simulated customer arrival times.
    - **Traffic Lights:** Testing light timings by simulating car build-up at an intersection.